

PROJEKTPRÄSENTATION

TravelMate Architektur & Datenfluss

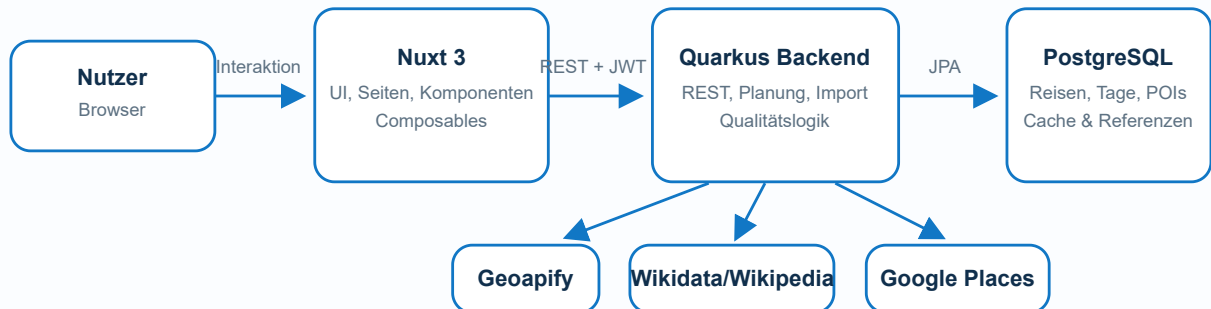
Wie aus einer Stadt, einem Zeitraum und persönlichen Interessen ein strukturierter, datenbasierter Reiseplan entsteht.

Kernidee: Das Frontend sammelt Präferenzen. Das Backend übernimmt Datenbeschaffung, Qualitätsbewertung, Planung und Speicherung.

Stack: Nuxt 3 · Java 21 / Quarkus · PostgreSQL · Docker Compose

1. Architektur in einem Satz

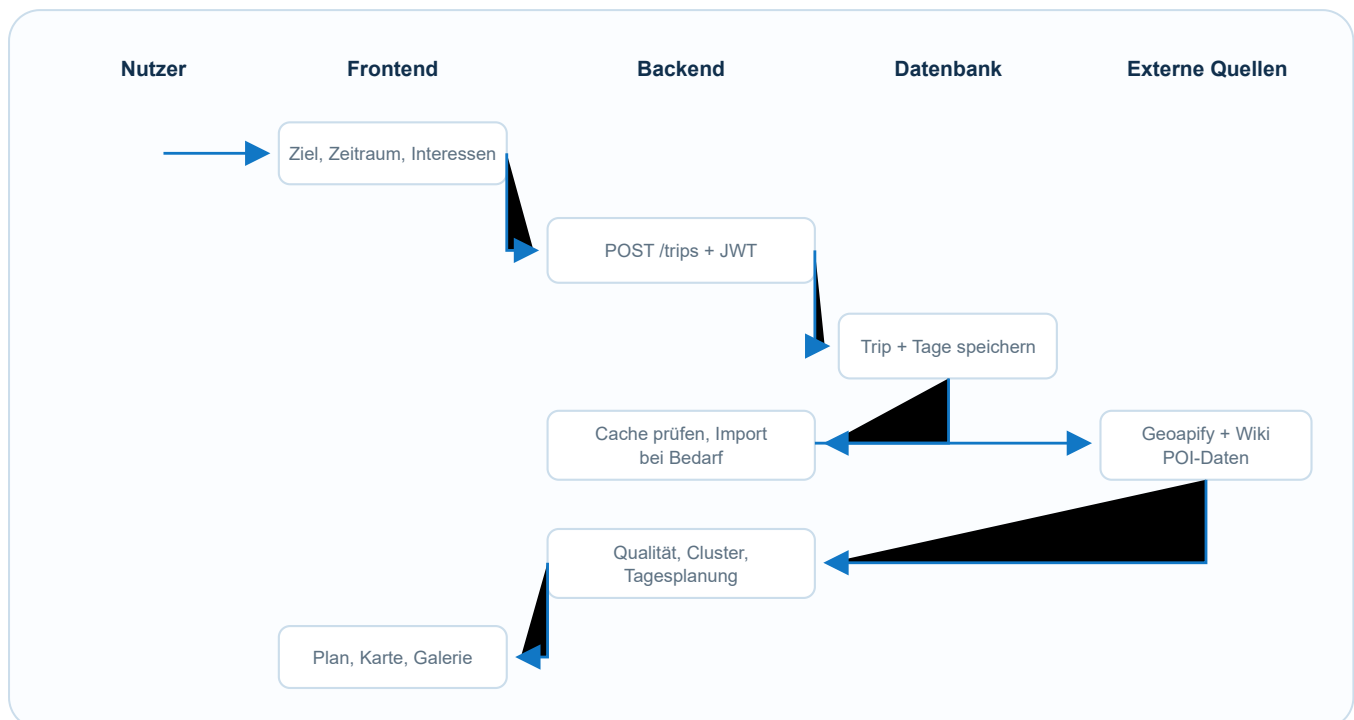
TravelMate nimmt Reisepräferenzen im Nuxt-Frontend entgegen, erstellt daraus über ein Java-/Quarkus-Backend einen regelbasierten Plan, reichert Orte bei Bedarf über externe Datenquellen an und speichert die Ergebnisse zentral in PostgreSQL.



2. Die Schichten des Projekts

Schicht	Aufgabe	Beispiele
Frontend	Eingaben, Darstellung und Interaktion	Interview, Kalender, Tagesplan, Karte, Galerie
Composables	Gemeinsame Frontend-Logik und API-Zugriffe	<code>useTripWorkspace</code> , <code>useTripImages</code> , <code>useLocationAutocomplete</code>
REST-API	Klare Schnittstelle zwischen Browser und Backend	<code>/trips</code> , <code>/locations</code> , <code>/activities</code>
Backend-Services	Validierung, Import, Qualitätsbewertung und Planung	<code>TripService</code> , <code>PlanningService</code> , <code>ActivitySyncService</code>
Datenbank	Dauerhafte Speicherung und Cache	Reisen, Tage, Aktivitäten, Bilder, externe Referenzen
Externe Quellen	Orte, Hintergrunddaten und Bilder	Geoapify, Wikidata, Wikipedia, Google Places

3. Datenfluss: Eine neue Reise planen



- 1 Der Nutzer wählt Ziel, Zeitraum, Interessen, Tempo und Tagesrhythmus.
- 2 Das Frontend sendet den Erstellungswunsch an das Backend.
- 3 Das Backend erstellt Reise und Reisetage und prüft den lokalen Ortsbestand.
- 4 Nur falls Daten fehlen oder veraltet sind, startet ein externer Import.
- 5 Die Planungslogik wählt passende, hochwertige und räumlich sinnvolle Aktivitäten.
- 6 Das Backend liefert den vollständigen Reiseplan als DTO zurück; Nuxt aktualisiert Karte, Liste und Galerie.

Wie ein API-Call formuliert ist

```

POST /trips
Authorization: Bearer <JWT>

{
  "city": "Kopenhagen", "daysCount": 3,
  "interestIds": [1, 4, 5],
  "startDate": "2026-07-10", "endDate": "2026-07-12",
  "planningDates": ["2026-07-10", "2026-07-11", "2026-07-12"],
  "pace": "BALANCED", "dayRhythm": "BALANCED",
  "country": "Dänemark", "countryCode": "DK",
  "latitude": 55.6761, "longitude": 12.5683
}

```

Bei einer flexiblen Reise bleiben konkrete Start- und Enddaten leer. Stattdessen werden Tagesanzahl und optional ein Wunschmonat übertragen.

TravelMate · Architektur und Datenfluss

4. Was passiert im Backend mit den Daten?

1. Request entgegennehmen

`TripResource` nimmt den REST-Request entgegen. `TripService` validiert die Daten und erstellt Reise sowie Reisetage.

2. Cache-Entscheidung

`ActivitySyncService` prüft für Stadt und Interessen Import-Version, Aktualität, Menge und räumliche Abdeckung.

3. Orte importieren

Geoapify liefert POIs. Ein Mapper überführt Rohdaten in ein internes Kandidatenmodell: Name, Adresse, Koordinaten, Kategorien und Referenzen.

4. Wissen anreichern

Wikidata ergänzt Referenzen und Bekanntheit. Wikipedia ergänzt, wenn verfügbar, Beschreibungstexte.

5. Qualität bewerten

Ungeeignete Orte werden ausgeschlossen. Übrig bleiben Kandidaten mit Kategorie-Fit, Bekanntheit, Datenqualität und sinnvoller räumlicher Lage.

6. Plan erzeugen

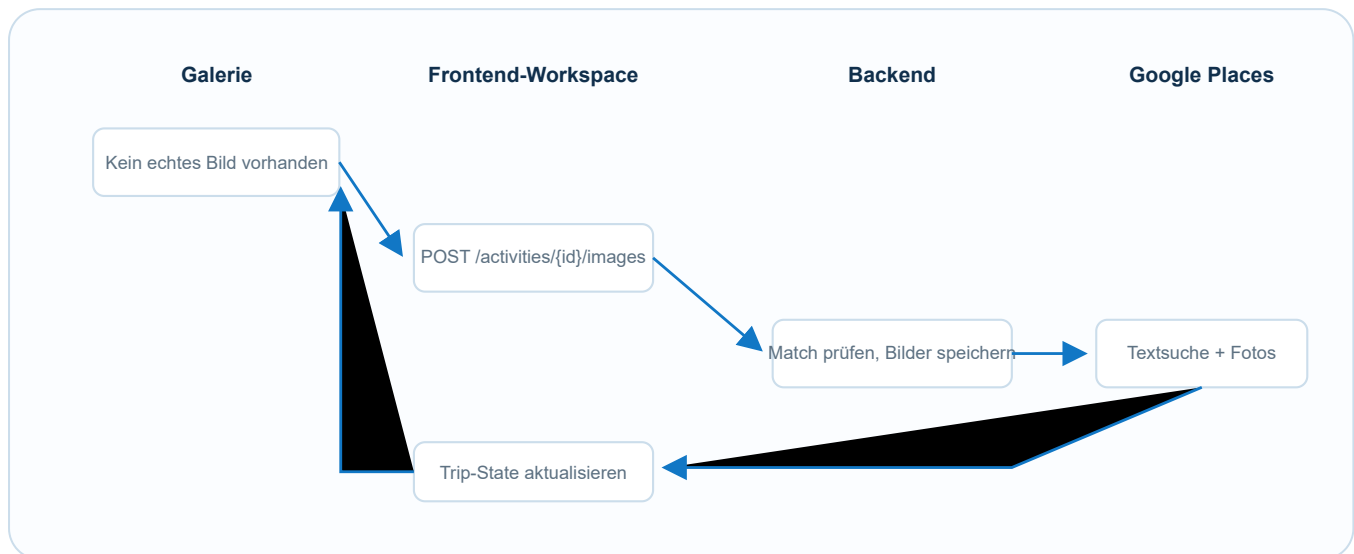
Quoten, Interessen, räumliche Cluster, Vielfalt, Tageszeit und Tempo werden kombiniert. So entstehen abwechslungsreiche Tagespläne.

Wann werden externe Quellen angefragt?

Zeitpunkt	Quelle	Zweck
Während der Stadteingabe	Geoapify Autocomplete	Stadtorschläge, Land, Koordinaten und Place-ID
Bei neuer Reise, wenn Cache nicht genügt	Geoapify Places	Aktivitätskandidaten für die gewählten Interessen
Direkt nach dem Import	Wikidata und Wikipedia	Relevanzsignale, Referenzen und Beschreibungen
Beim Öffnen von Highlights	Wikidata + Wikimedia Pageviews	Touristisch relevante Highlights, mit Cache
Beim Anzeigen einer Aktivität ohne Bild	Google Places	Nachträgliche Bildanreicherung

5. Bilddatenfluss – bewusst nachgelagert

Bilder sind kein Blocker für die Reiseerstellung. Der Plan ist sofort nutzbar; echte Bilder werden gezielt nachgeladen.



- Der aktive Tag wird zuerst behandelt; anschließend werden die zwei folgenden Tage vorgeladen.
- Ein gemeinsamer In-Flight-Status verhindert doppelte Bildanfragen für dieselbe Aktivität.
- Das Backend speichert bis zu drei Google-Bildreferenzen pro Aktivität.
- Beim tatsächlichen Rendern löst ein sicherer Backend-Endpunkt die Foto-URL auf und leitet den Browser temporär weiter.
- Falls kein Match zustande kommt oder ein Bild temporär nicht erreichbar ist, bleibt ein Kategorie-Platzhalter sichtbar.

6. Planänderungen ohne Neuplanung

Nutzeraktion	API-Aufruf	Reaktion im Backend
Aktivität löschen	<code>DELETE /trips/{id}/days/{dayId}/activities/{itemId}</code>	Eintrag entfernen und Tagesplan aktualisieren
Aktivität ersetzen	<code>PUT /trips/{id}/days/{dayId}/activities/{itemId}</code>	Passende Alternative auswählen
Aktivität neu generieren	<code>POST .../regenerate</code>	Beste freie Alternative suchen
Aktivitäten oder Tage verschieben	<code>PUT /trips/{id}/schedule</code>	Reihenfolge, Zieltag und Zeiten speichern
Zeitfenster ändern	<code>PUT .../availability</code>	Tageszeiten validieren und neu berechnen

7. Präsentationsfazit

„TravelMate trennt bewusst zwischen schneller Nutzeroberfläche und datenintensiver Hintergrundlogik. Das Frontend sammelt Präferenzen und zeigt Ergebnisse. Das Backend entscheidet, ob Ortsdaten bereits lokal vorhanden sind, importiert bei Bedarf neue POIs, bewertet sie nach Qualität und räumlicher Eignung und baut daraus einen Tagesplan. Zusätzliche Daten wie Bilder und Highlights werden erst nachgeladen, wenn sie für die aktuelle Ansicht wirklich relevant sind.“

Die drei Kernbotschaften

1. **Schnell:** Plan und Interaktion bleiben nutzbar, während Zusatzdaten im Hintergrund nachgeladen werden.
2. **Qualitätsorientiert:** Nicht jeder gefundene Ort wird genutzt; TravelMate filtert, bewertet und plant regelbasiert.
3. **Effizient:** Lokale Persistenz und Caching reduzieren wiederholte externe API-Anfragen.

Technische Grundlage: Nuxt 3, Java 21/Quarkus, PostgreSQL, Docker Compose. Externe Daten: Geoapify, Wikidata, Wikipedia, Google Places und OpenStreetMap-Kartenkacheln.